

Задача A1

Анализ строковых сортировок

Задача	ID ссылки
A1m	375963079
A1q	375963355
A1r	375966130
A1rq	375969103

Выполнил: Коробов Вячеслав
Группа: 246

Реализации классов StringGenerator и StringSorter также находятся в репозитории по ссылке <https://github.com/StudentSK0/comparison-of-sorting-algorithms>.

1. Эмпирический анализ стандартных алгоритмов

Для стандартных алгоритмов QUICKSORT и MERGESORT были измерены две характеристики: среднее время выполнения и количество посимвольных сравнений. Тестирование проводилось на трех наборах данных: *случайных*, *обратных* и *почти отсортированных* строках. Для каждого набора использовались подмассивы размеров от 100 до 3000 строк с шагом 100.

1.0.1. Время выполнения

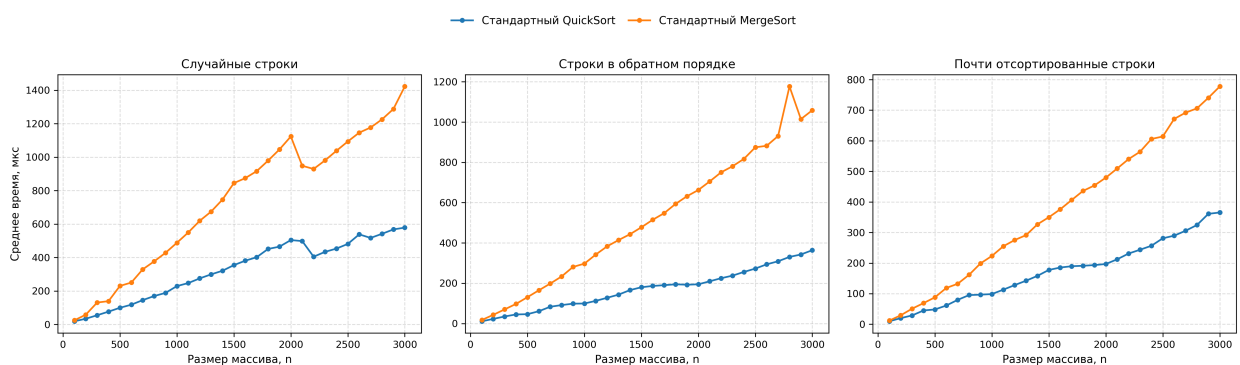


Рис. 1: Время выполнения стандартных алгоритмов сортировки на трех наборах данных

По времени выполнения QUICKSORT оказался быстрее MERGESORT на всех трех типах данных. Это связано с тем, что QUICKSORT сортирует массив преимущественно на месте, тогда как MERGESORT использует дополнительный буфер и выполняет больше операций перемещения элементов при слиянии.

На обратно отсортированных и почти отсортированных данных резкого ухудшения алгоритма QUICKSORT не наблюдается, поскольку в реализации опорный элемент выбирался из середины текущего фрагмента массива. Такой способ выбора опорного элемента снижает риск перехода к худшему случаю на упорядоченных данных.

1.0.2. Количество посимвольных сравнений

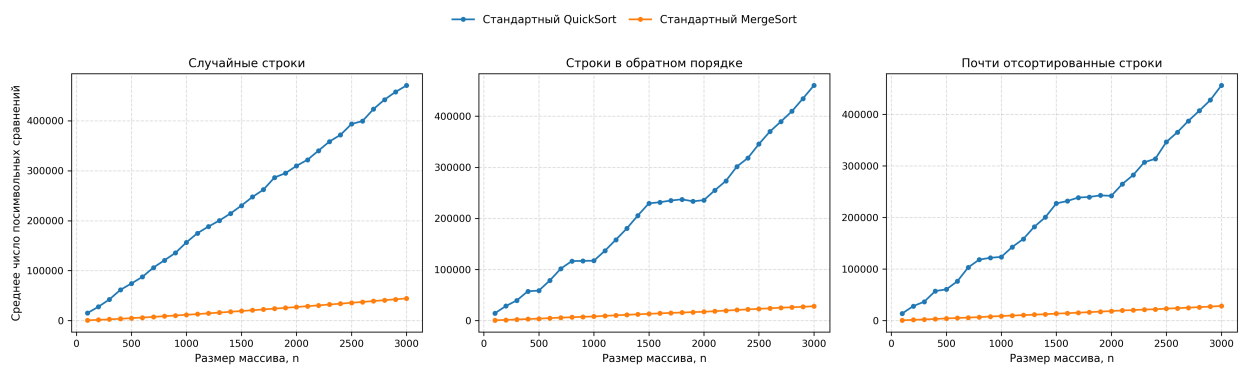


Рис. 2: Количество посимвольных сравнений стандартных алгоритмов сортировки на трех наборах данных

По количеству посимвольных сравнений результат отличается от результата по времени: QUICKSORT выполняет заметно больше посимвольных сравнений, чем MERGESORT. Это объясняется тем, что в процессе разбиения массива элементы многократно сравниваются с опорными значениями. MERGESORT выполняет сравнения более регулярно при слиянии уже отсортированных частей, поэтому число посимвольных сравнений оказывается ниже.

Таким образом, меньшее число посимвольных сравнений не всегда означает меньшее время выполнения. Для MERGESORT существенную роль играют накладные расходы на дополнительную память и копирование строк при слиянии, поэтому по времени он уступает QUICKSORT, несмотря на меньшее количество посимвольных сравнений.

1.0.3. Дополнительный набор строк с общим префиксом

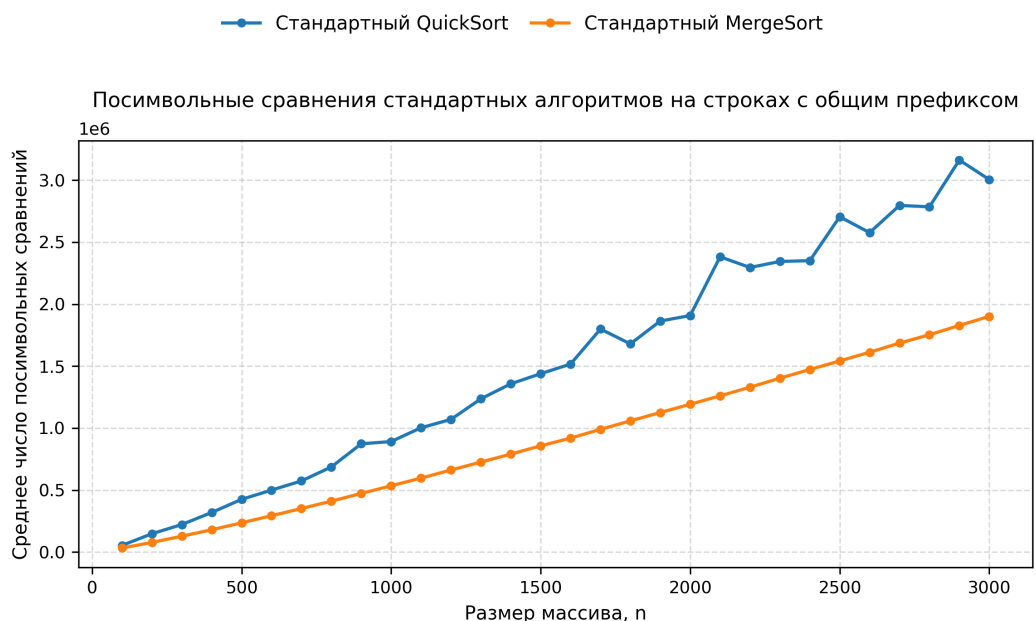


Рис. 3: Количество посимвольных сравнений стандартных алгоритмов на строках с длинным общим префиксом

Дополнительный набор строк с общим префиксом показывает ключевую особенность строковой сортировки. Оба стандартных алгоритма многократно заново сравнивают совпадающие начальные части строк. Поэтому при наличии длинного общего префикса количество посимвольных сравнений существенно возрастает.

Этот результат показывает, что для строк стандартная оценка сложности сортировки на основе сравнений недостаточна сама по себе. Необходимо учитывать не только количество сравнений элементов массива, но и стоимость каждого сравнения строк, зависящую от длины общего префикса.

2. Эмпирический анализ адаптированных алгоритмов

Рассмотрены четыре специализированных алгоритма строковой сортировки: тернарный STRING QUICKSORT, алгоритм LCP MERGESORT, сортировка MSD RADIX SORT и ее версия с переключением на STRING QUICKSORT при размере фрагмента меньше мощности алфавита. Для сопоставимости также были включены стандартные QUICKSORT и MERGESORT. В стандартном QUICKSORT и тернарном STRING QUICKSORT использовался одинаковый способ выбора опорного элемента: элемент из середины текущего фрагмента массива.

Для MSD RADIX SORT значение счётчика посимвольных операций – это не количество попарных сравнений строк, а количество обращений к символам при распределении строк по корзинам.

2.1. Сравнение времени выполнения

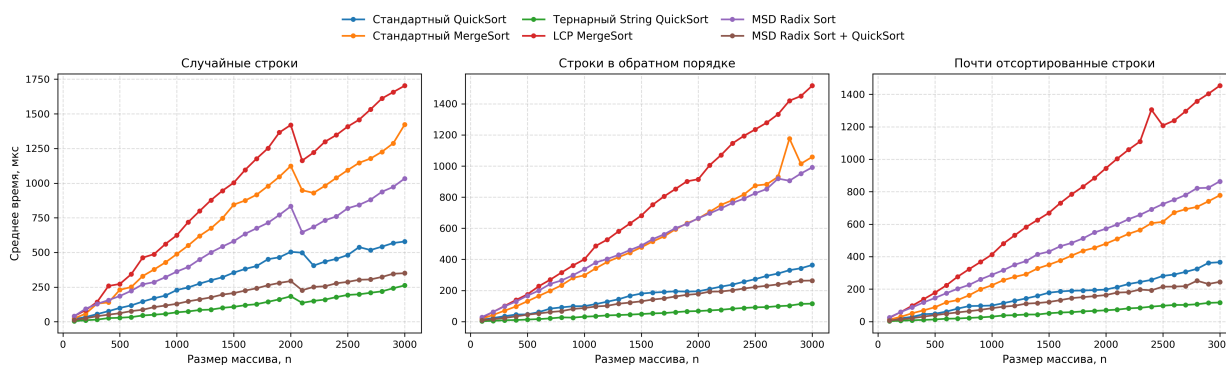


Рис. 4: Время выполнения стандартных и адаптированных алгоритмов

По времени выполнения лучшим алгоритмом оказался тернарный STRING QUICKSORT. Он стабильно быстрее стандартного QUICKSORT, поскольку не выполняет полное лексикографическое сравнение строк при каждом разбиении, а работает с символом на текущей глубине.

Несмотря на уменьшение повторных сравнений префиксов, LCP MERGESORT имеет дополнительные накладные расходы на хранение и обновление LCP-информации при слиянии. Поэтому на строках без специально выраженных длинных общих префиксов эти расходы оказываются существенными.

MSD RADIX SORT тоже не всегда выигрывает по времени. Это происходит из-за высокой стоимости распределения по корзинам и рекурсивной обработки большого числа фрагментов. Вариант с переключением на STRING QUICKSORT работает быстрее чистого MSD RADIX SORT, так как уменьшает накладные расходы на малых подмассивах.

2.2. Сравнение количества посимвольных сравнений

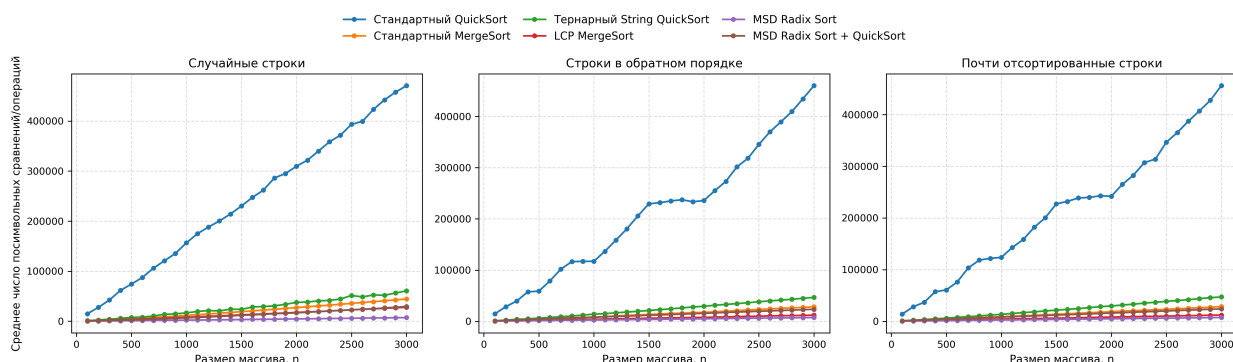


Рис. 5: Количество посимвольных сравнений и операций

По количеству посимвольных сравнений адаптированные алгоритмы значительно превосходят стандартные. Особенно заметно отличие от стандартного QUICKSORT, который многократно сравнивает строки с опорными элементами и повторно просматривает одинаковые начальные части строк.

Минимальное число посимвольных операций показывают варианты MSD RADIX SORT. Это соответствует идее алгоритма: строки не сравниваются попарно, а распределяются по символам начиная со старшей позиции. LCP MERGESORT также уменьшает число повторных сравнений по сравнению со стандартным MERGESORT, так как использует уже известную длину общего префикса при слиянии.

2.3. Дополнительный набор строк с общим префиксом

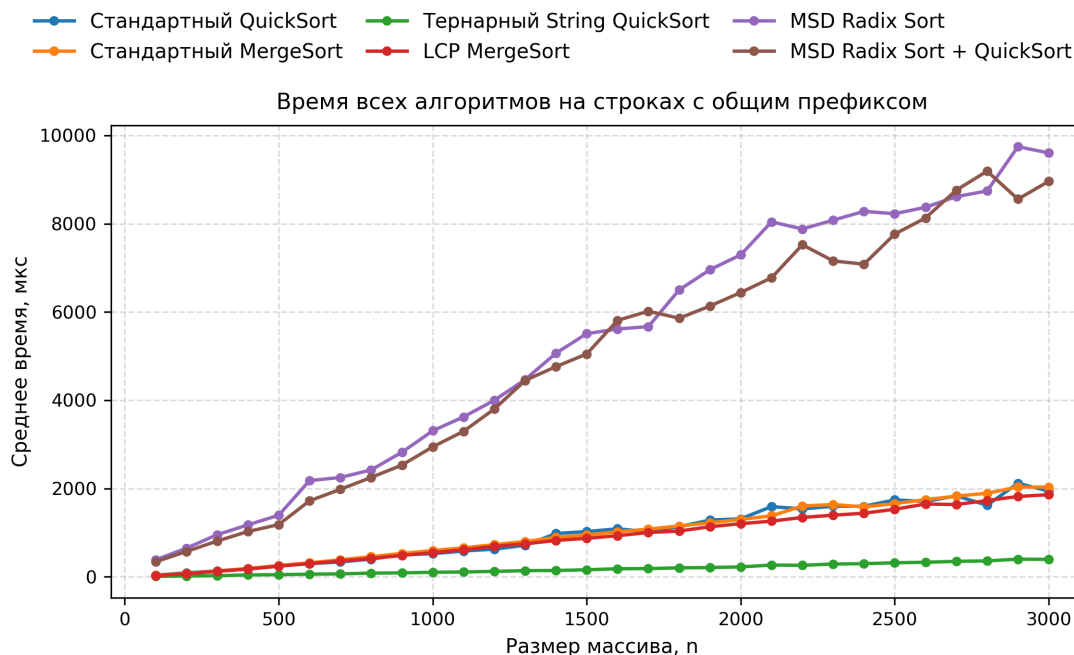


Рис. 6: Время выполнения всех алгоритмов на строках с длинным общим префиксом

На строках с длинным общим префиксом преимущество тернарного STRING QUICKSORT становится наиболее заметным. Алгоритм эффективно обрабатывает группы строк с одина-

ковыми символами на текущей позиции и переходит к следующей позиции только внутри равной группы.

При этом MSD RADIX SORT и его вариант с переключением выполняются медленнее, несмотря на малое число посимвольных операций. Это показывает, что практическое время определяется не только числом обращений к символам, но и накладными расходами на создание и обработку корзин.

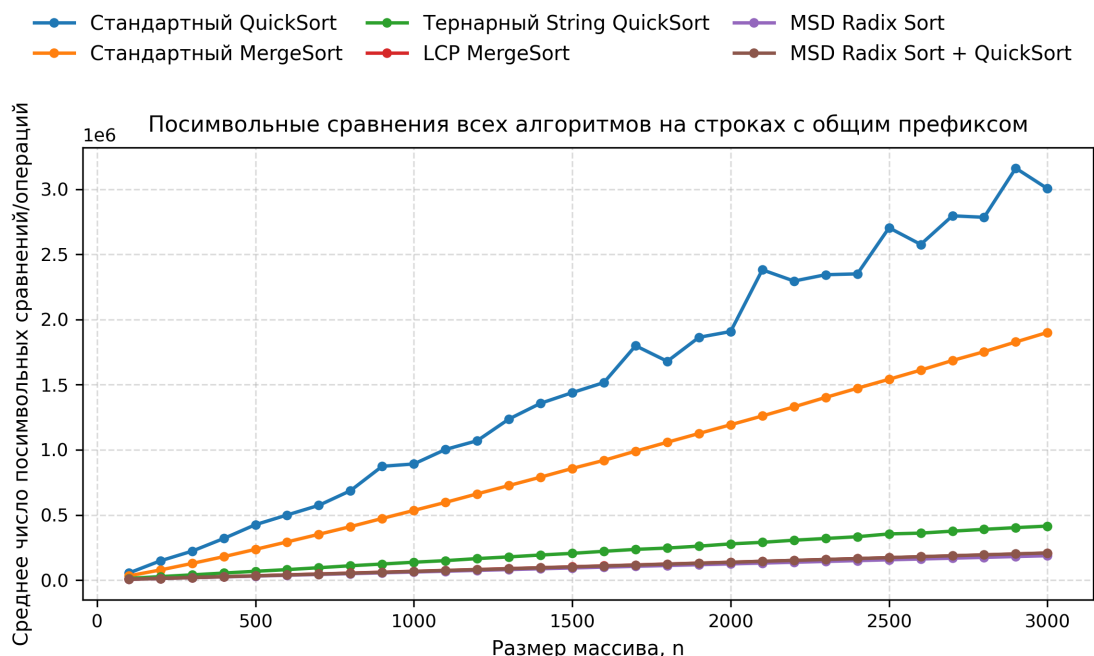


Рис. 7: Количество посимвольных сравнений и операций на строках с длинным общим префиксом

На наборе с общим префиксом стандартные алгоритмы выполняют существенно больше посимвольных сравнений, так как при каждом сравнении заново просматривают совпадающую начальную часть строк. Адаптированные алгоритмы уменьшают этот эффект: STRING QUICKSORT группирует строки по текущему символу, LCP MERGESORT использует информацию о длине общего префикса, а MSD RADIX SORT распределяет строки по символам без попарного сравнения.

2.4. Итоговое сравнение с базовым алгоритмом

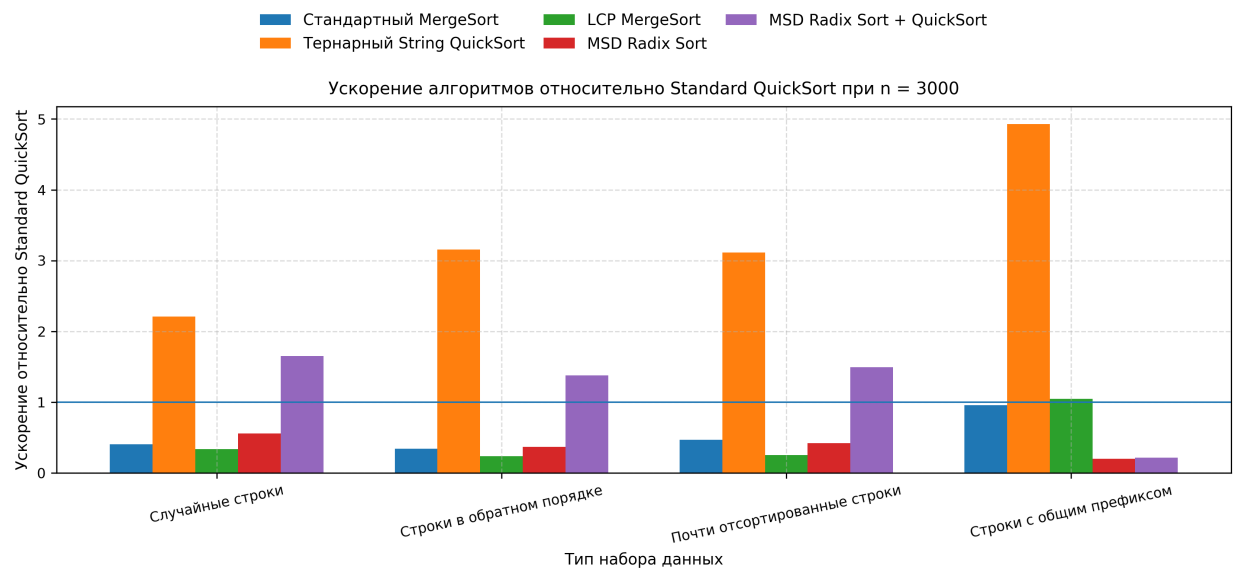


Рис. 8: Ускорение алгоритмов относительно стандартного QUICKSORT при $n = 3000$

Итоговое сравнение при $n = 3000$ показывает, что наибольшее ускорение относительно стандартного QUICKSORT даёт тернарный STRING QUICKSORT. На трех наборах данных он работает в несколько раз быстрее базового алгоритма, а на строках с общим префиксом его преимущество становится максимальным.

Вариант MSD RADIX SORT с переключением на STRING QUICKSORT быстрее базового QUICKSORT на трех наборах данных, но проигрывает на строках с общим префиксом из-за накладных расходов. Чистый MSD RADIX SORT показывает малое число посимвольных операций, но по времени уступает из-за высокой стоимости корзин. LCP MERGESORT уменьшает число сравнений, однако по времени не является лучшим из-за дополнительных операций с LCP-значениями.